

Name/NetID: Nathan Kariuki / nkari011

Session Date: May 19, 2026 (1-2 hours)

Objective: Put together the full speedup table, answer the discussion questions, and organize the repo for submission.

Attempts made: Needed the Phase 2b serial naive baseline times to compute speedups for the 2c table. Pulled those from the Phase 2b journal. Serial naive baselines were approximately BFS 1.25s, SSSP 13.6s, CC 22.4s based on the Phase 2b speedup table numbers.

Organized the repo by moving all code files into their respective files

Solutions/Workarounds: No major issues this session. Mostly analysis and writeup work.

Commits: journal: add all session files

Experiments/Analysis: Full speedup table using Phase 2b serial naive push as the 1x baseline:

Algorithm / 2b Dense parallel 4t (lock K=1024) / 2c Dense parallel 4t (atomic) / 2c Sparse parallel 4t (atomic)

BFS / 1.56x / 1.57x / 1.52x SSSP / 1.20x / 2.26x / 2.22x CC / 0.57x / 7.86x / 12.43x

Discussion question 1: Which algorithm benefits most from dropping the lock?

CC benefits the most by far. In Phase 2b CC dense parallel with locks was actually slower than serial at 0.57x. With atomics it jumped to 7.86x. The reason is that CC pushes to both out-edges and in-edges for every active vertex, so each vertex has twice the contention compared to BFS or SSSP. With locks, threads were constantly blocking each other on high-degree vertices. The CAS avoids that entirely since threads never block, they just retry or move on.

BFS barely changed between locked and atomic. BFS does a single one-shot CAS per neighbor and each vertex is discovered at most once so there was never much contention to begin with. The lock overhead still existed but BFS finishes in 14 rounds on LiveJournal so the total lock overhead was small.

SSSP improved more noticeably than BFS. SSSP has more contention than BFS since distances can be relaxed multiple times per round, so dropping the lock helped more.

Discussion question 2: Sparse vs dense atomic?

BFS: dense and sparse are almost identical on LiveJournal because the frontier grows to over 2M vertices by round 5. At that point iterating the sparse list costs about as much as scanning the full bitmap. On roadNet sparse is 18x faster because the frontier stays small for hundreds of rounds.

SSSP: sparse was slightly slower than dense on LiveJournal. SSSP runs for 41 rounds and keeps a large frontier throughout so the sparse overhead from managing the worklist and dedup flags adds up without much payoff. On roadNet sparse was about 2.5x faster.

CC: sparse wins on both datasets. On LiveJournal CC sparse was 1.58x faster than CC dense. The frontier drops off fast after round 1 so the sparse list pays off quickly. On roadNet sparse was about 2.15x faster.

Discussion question 3: Worklist size per round and ideas for bonus.

BFS frontier on LiveJournal peaks at 2.1M in round 5 then shrinks fast. Early and late rounds are sparse, middle rounds are dense. This is exactly the pattern that direction-optimizing BFS exploits. Switch to sparse when the frontier is small, switch to dense when it grows large. The threshold from the spec is roughly when $|F|$ exceeds $n / \text{average_degree}$.

SSSP frontier grows slowly over 41 rounds and stays large for most of them, peaking around 3.4M. The sparse representation never gets much benefit here because there are very few rounds where $|F|$ is truly small relative to n .

CC frontier starts at 4.8M in round 0 and drops to near zero by round 6. Only the last few rounds benefit from sparse. The first round is always dense by construction so direction-optimizing CC would not help much on LiveJournal.

On roadNet all three algorithms keep small frontiers for most of their rounds which is why sparse dominates there.

Learning outcome: The speedup from dropping locks depends on how much contention the algorithm actually generates. CC has the most contention because it accesses both edge directions, so it benefits the most from atomics. The sparse vs dense tradeoff comes down to frontier size relative to n and that varies a lot between graph types. A direction-optimizing variant that switches each round based on frontier size would get the best of both worlds.